Lab 2

A. Introduction

Objectives

At the end of this lab you should be able to:

- Use direct addressing mode of accessing data in memory
- Use indirect addressing mode of accessing data in memory
- Write a subroutine and call it
- Pass parameters to a subroutine

B. Processor (CPU) Simulators

The computer architecture tutorials are supported by simulators, which are created to underpin theoretical concepts normally covered during the lectures. The simulators provide visual and animated representation of mechanisms involved and enable the students to observe the hidden inner workings of systems, which would be difficult or impossible to do otherwise. The added advantage of using simulators is that they allow the students to experiment and explore different technological aspects of systems without having to install and configure the real systems.

C. Basic Theory

The programming model of computer architecture defines those low-level architectural components, which include the following

- CPU instruction set
- CPU registers
- Different ways of addressing instructions and data in instructions, i.e. addressing modes such as direct and indirect addressing.

They also define interactions between the above components. It is this low-level programming model which makes programmed computations possible.

D. Simulator Details

You can find the simulator details relevant to this tutorial in Programming Model 1 tutorial. As a result, these are not repeated here except the new information on how to view and access program data memory view; this tutorial requires access to this part of the simulator. You will find the details below.

Program data memory view

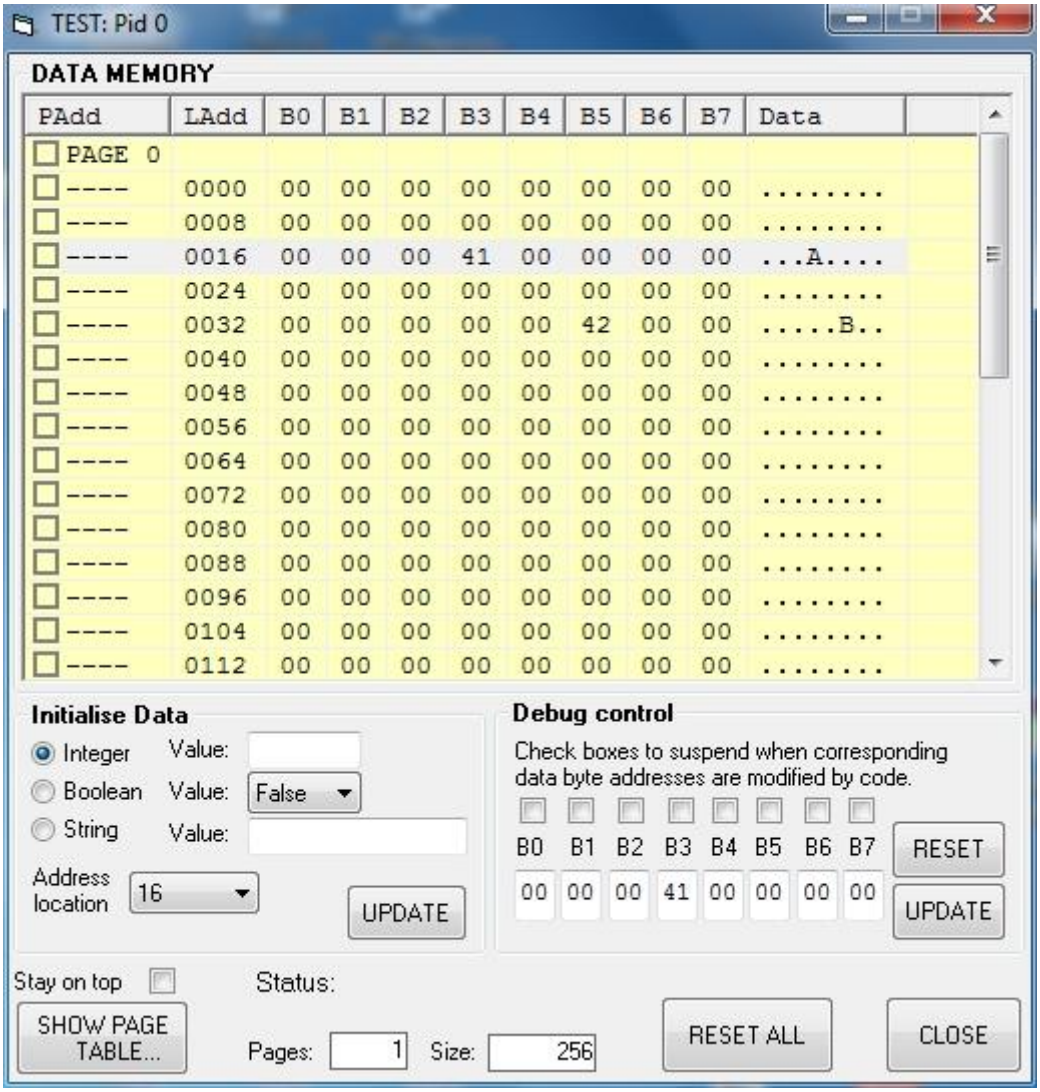

Image 1 - Program data memory view

The CPU instructions that access that part of the memory containing data can write or read the data in addressed locations. This data can be seen in the memory pages window shown in Image 1 above. You can display this window by clicking the **SHOW PROGRAM DATA MEMORY…** button in the main CPU window. The **Ladd** (logical address) column shows the starting address of each line in the display. Each line of the display represents 8 bytes of data. Columns **B0** through to **B7** represent bytes 0 to 7 on each line. The **Data** column shows the displayable characters corresponding to the 8 bytes. Those bytes that correspond to non-displayable characters are shown as dots. The data bytes are displayed in hex format only. For example, in Image 1, there are non-zero data bytes in address locations 19 and 37. These data bytes correspond to displayable characters capital A and B.

To manually change the values of any bytes, first select the line(s) containing the bytes. Then use the information in the **Initialize Data** frame to modify the values of the bytes in the selected line(s) as **Integer**, **Boolean** or **String** formats. You need to click the **UPDATE** button to make the change.

E. Lab Exercises - Investigate and Explore (80 points)

Enter the instructions you create in order to answer the questions in the blank boxes. Refer to Appendix at the end of this document to find the details on the desired instructions. You are expected to execute the instructions you created on the simulator in order to verify your answers.

A. Instructions for writing to and reading from memory (RAM):

1. Locate the instruction that **stores** a byte in program data memory and use it to store number 65 in memory address location 20 (this uses **memory direct** addressing method). **3 points**

2. Move number 51 into register R04. Use the store instruction to **store** the contents of R04 in program data memory location 21 (this uses **register direct** addressing method). **3 points**

3. Move number 22 into register R04. Use this information to indirectly **store** number 59 in program data memory (hint: you will need to use the '@' prefix for this – see the list of instructions in appendix) - (this uses **register indirect** addressing method).**3 points**

4. Locate the instruction that **loads** a byte from program data memory into a register. Use this to load the number in memory address 22 into register R10. **3 points**

5. Write a loop in which 10 numbers from 41 to 50 are written in program data memory starting from memory address 24 (hint: use **register indirect** addressing where register R04 indirectly represents memory address to write to and increment it to address increasing memory locations).**10 points**

6. Manually initialise part of program data memory starting from address 40 with string "CPU INSTRUCTIONS RULE" (see section D above on how to do this). Write a loop in which this string is copied to another part of the program data memory starting from address 64. **10 points**

B. Instructions for calling subroutines and passing parameters to subroutines:

1. Add the following code and run it starting from the first MOV instruction (to do this you need to first select this instruction and then click on the RUN button).

   NOTE:
   Ask your tutor how to add labels to your code. A Label represents the address of the instruction immediately following it. For example, 'Label2' below represents the address of the MOV instruction following it. Labels are used by the jump instructions by putting a '$' in front of the label name, e.g. the 'JMP $Label2' instruction will jump to the instruction at address represented by the label 'Label2'.

```
Label2
MOV #16, R03
MOV #h41, R04
Label3
STB R04, @R03
ADD #1, R03
ADD #1, R04
CMP #h4F, R04
JNE $Label3
HLT
```

NOTE:
This code stores numbers hex 41 to hex 4F starting from program memory address 16. These numbers are ASCII codes for displayable characters of the alphabet. You need to understand how this is done by this code in order to benefit from these exercises.

   a. Make a note of what you see in the program's data area after the program stops running:
   5 points

   b. Suggest what the significance of @ in **@R03** might be:
   5 points

2. Add the following subroutine calling code but do **NOT** run it yet:

```
MSF
CAL $Label2
HLT
```

   Now convert the code in (1) above into a subroutine by simply replacing the **HLT** instruction with the **RET** instruction.

a. Make a note of the contents of the **PROGRAM STACK** after the instruction **MSF** is executed (you can execute this instruction by simply double-clicking on it):**3 points**

b. Make a note of the contents of the **PROGRAM STACK** after the instruction **CAL** is executed (you can execute this instruction by simply double-clicking on it):**3 points**

c. What is the significance of the additional information on the stack after executing the **CAL** instruction?**2 points**

3. Let's make the above subroutine a little more flexible. Suppose we wish to change the number of characters stored when calling the subroutine. Modify the calling code in (2) as below:

```
MSF
PSH #h60        ← puts the number hex 60 on top of the stack
CAL $Label2
HLT
```

a. Now modify the subroutine code in (1) as below and run the above calling code (starting from the **MSF** instruction). Pay particular attention to the behaviour of the stack:

```
Label2
MOV #16, R03
MOV #h41, R04
POP R05         ← puts the number on top of the stack in R05
Label3
STB R04, @R03
ADD #1, R03
ADD #1, R04
CMP R05, R04    ← compares R05 with R04
JNE $Label3
RET
```

b. Add a second parameter to the above code that can provide different starting addresses for the data transfer to memory and test it on the simulator. Write the modified code below:

10 points

4. Write a subroutine that takes two numbers as parameters, adds them and returns the result in register R00, i.e. when the subroutine is exited the result is available in R00 to the rest of the program. Test it on the simulator by writing the calling instructions that include passing two numbers on the stack. Copy the code below (including the calling instructions):**10 points**

5. Write a subroutine that takes two numbers as parameters, compares them and returns the higher of the two as the result in register R00 (consider what should happen if the two numbers are the same). Test it on the simulator by writing the calling instructions that include passing two numbers on the stack. Copy the subroutine code alone below:**10 points**